

Optimization Algorithm Visualizer - Complete Documentation

Table of Contents

- [1. Project Overview](#)
 - [2. Architecture](#)
 - [3. Technology Stack](#)
 - [4. Backend System](#)
 - [4.1 Core Optimization Framework](#)
 - [4.2 Server Endpoints](#)
 - [5. Frontend System](#)
 - [5.1 HTML Structure](#)
 - [5.2 JavaScript Application](#)
 - [5.3 Styling](#)
 - [6. Optimization Algorithms](#)
 - [7. Line Search Methods](#)
 - [8. Test Functions](#)
 - [9. User Guide](#)
 - [10. API Reference](#)
 - [11. Infrastructure & Cold Start Handling](#)
 - [12. File Structure Summary](#)
 - [13. Troubleshooting](#)
 - [14. References & Resources](#)
-

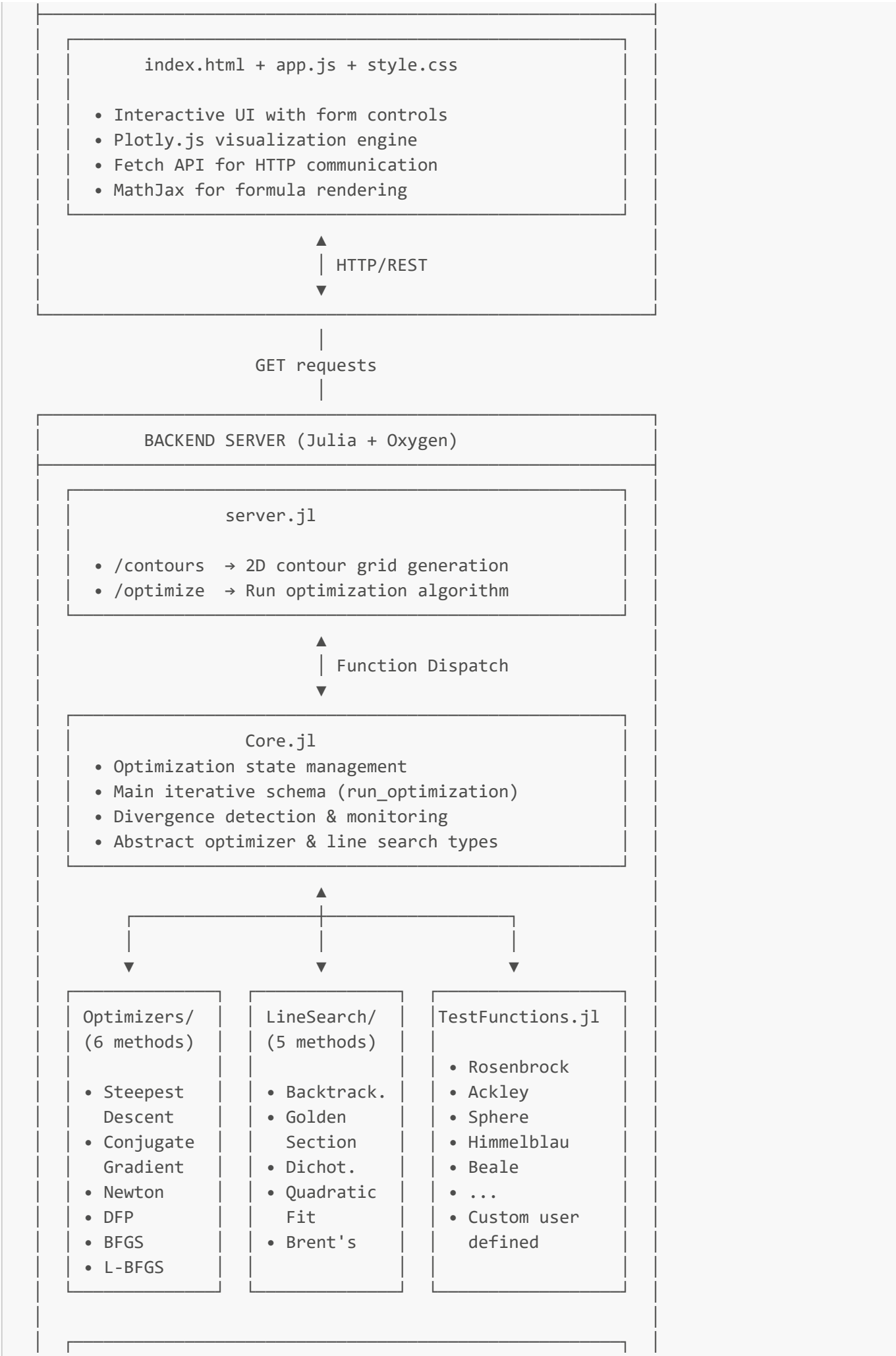
1. Project Overview

The **Optimization Algorithm Visualizer** is an interactive web application designed to demonstrate various mathematical optimization algorithms. Users can:

- **Select from 10 built-in test functions** or define custom functions
 - **Choose among 6 optimization methods** with multiple variants
 - **Combine with 5 different line search strategies** including the choice of auto-bracketing
 - **Configure optimization parameters** including dimensionality, starting points, and termination criteria
 - **Visualize optimization trajectories** on 2D contour plots
 - **Monitor convergence behavior** through evolution plots (function value, gradient norm, step size, step distance)
 - **Compare against ground truth** computed via Julia's `Optim.jl` library
-

2. Architecture



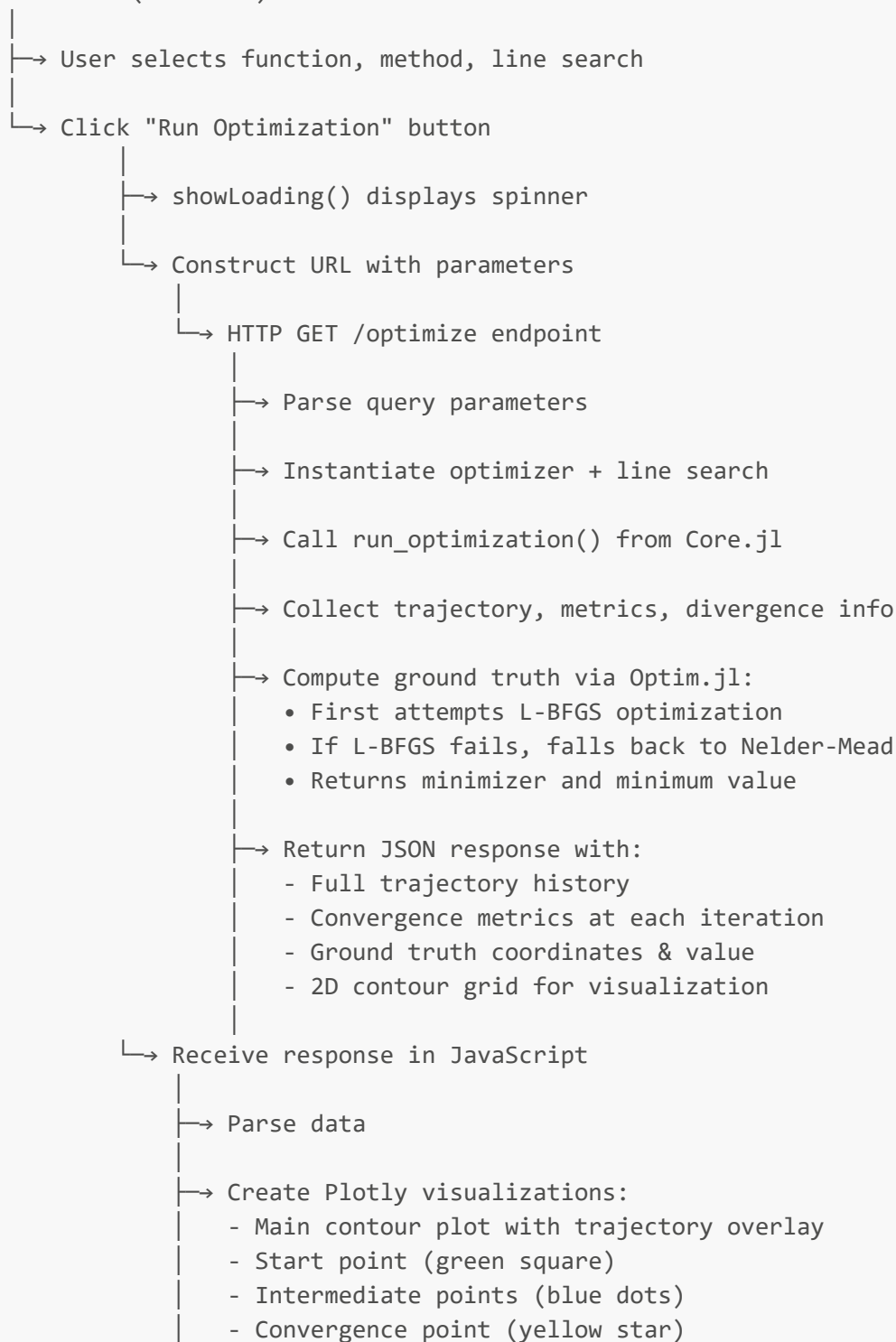


External Libraries

- ForwardDiff.jl (Automatic differentiation)
- LinearAlgebra (Matrix operations)
- Optim.jl (Ground truth via L-BFGS)
- Oxygen.jl (HTTP server framework)

Component Interaction Flow

User Interaction (Frontend)



```

    | - Ground truth (black X)
    |
    |→ Display 4 evolution plots:
    |   -  $f(x_k)$  evolution
    |   -  $||\nabla f(x_k)||$  evolution
    |   -  $\alpha_k$  step size evolution
    |   -  $||\Delta x_k||$  step distance evolution
    |
    |→ hideLoading() and show results panel

```

3. Technology Stack

Backend

- **Language:** Julia (scientific computing language)
- **Web Framework:** [Oxygen.jl](#) (lightweight HTTP server)
- **Math Libraries:**
 - [ForwardDiff.jl](#) - Automatic differentiation for gradients & Hessians
 - [LinearAlgebra](#) - Matrix operations
 - [Optim.jl](#) - Ground truth optimization

Frontend

- **HTML5** - Markup structure
- **CSS3** - Styling with responsive layout
- **JavaScript (ES6+)** - Client-side logic and interactivity
- **Visualization:** [Plotly.js v2.27.0](#) - Interactive charts
- **Math Rendering:** [MathJax v3](#) - LaTeX formula display

Infrastructure

- **Hosting:** Render.com (free tier)
- **Resource Constraints:** Limited to 0.1 CPU and 512MB RAM on free tier
 - This results in slower performance, especially for high-dimensional problems or complex functions
 - Initial requests may take 30-120 seconds (cold start after 15 minutes of inactivity)
 - Subsequent requests after warm start are typically faster
- **API Base URL:** <https://optimization-app-wkcn.onrender.com>
- **CORS Handling:** Backend includes Access-Control-Allow-Origin headers allowing cross-origin requests from any domain
 - This enables the frontend (running on a different domain) to communicate with the backend via fetch API

Dependencies

See [backend/Project.toml](#):

```
ForwardDiff = "f6369f11-7733-5829-9624-2563aa707210"
HTTP = "cd3eb016-35fb-5094-929b-558a96fad6f3"
Optim = "429524aa-4258-5aef-a3af-852621145aeb"
Oxygen = "df9a0d86-3283-4920-82dc-4555fc0d1d8b"
```

4. Backend System

4.1 Core Optimization Framework (Core.jl)

The backend employs an abstract type hierarchy to support extensible algorithm design. This object-oriented approach allows different optimizer and line search implementations to be swapped without changing the core algorithm structure.

Abstract Base Types

Julia uses abstract types to define interfaces that concrete implementations must follow:

```
abstract type AbstractOptimizer end      # Interface for optimization methods
abstract type AbstractLineSearch end    # Interface for line search strategies
```

Each concrete optimizer (SteepestDescent, BFGS, etc.) and line search method (Backtracking, BrentsMethod, etc.) must be concrete subtypes of these abstracts. The core algorithm then dispatches to the appropriate implementation via Julia's multiple dispatch system.

Optimization State

```
mutable struct OptimizationState
    x::Vector{Float64}           # Current position
    gradient::Vector{Float64}    # Current gradient  $\nabla f(x)$ 
    inverse_hessian::Matrix{Float64} # Approximate or exact inverse Hessian
    history::Vector{Any}         # For LBFGS: stores (s, y,  $\rho$ ) triplets
end
```

Divergence Tracking

```
mutable struct DivergenceInfo
    diverged::Bool              # Whether divergence was detected
    reason::String              # Description of divergence
    iteration::Int              # When it occurred
    grad_norm::Float64          # Gradient magnitude at divergence
    f_value::Float64            # Function value at divergence
end
```

Key Constants & Thresholds

```
const GRAD_NORM_THRESHOLD = 1e8           # Gradient explosion
const F_VALUE_THRESHOLD = 1e20           # Function value explosion
const MIN_STEP_SIZE = 1e-16             # Step size collapse
```

Main Optimization Loop

```
function run_optimization(f, ∇f, x0::Vector{Float64},
    method::AbstractOptimizer,
    linesearch::AbstractLineSearch;
    max_iter=2000,
    term_criterion="gradient",
    tol=1e-4)
```

Algorithm:

- 1. Initialize state:
 - Position: $x_k \leftarrow x_0$
 - Gradient: $\nabla f_k \leftarrow \nabla f(x_0)$
 - Inverse Hessian approximation: $W_k \leftarrow I$ (identity matrix of dimension $n \times n$)
- 2. For each iteration $k = 1$ to max_iter :
 - Check if termination criterion is satisfied:
 - If yes, exit loop and return converged solution
 - If no, continue to next step
 - Compute search direction: d_k via `compute_direction(method, state)`
 - Perform line search: α_k via `compute_step_size(linesearch, f, ∇f, state, d_k)`
 - Update position: $x_{\{k+1\}} \leftarrow x_k + \alpha_k \cdot d_k$
 - Update gradient: $\nabla f_{\{k+1\}} \leftarrow \nabla f(x_{\{k+1\}})$
 - Check for divergence via `check_divergence(...)`
 - If diverged, stop and report divergence reason
 - Update method-specific state (Hessian approximations, CG coefficients, LBFGS history, etc.)
- 3. Return trajectory history, step size history, and divergence information

Divergence Detection

The `check_divergence()` function monitors for algorithmic failures:

Condition	Threshold
NaN/Inf in gradient	Any
NaN/Inf in $f(x)$	Any
NaN/Inf in position	Any
Gradient norm explosion	$\ \nabla f\ > 1e8$

Condition	Threshold
Function value explosion	$ f > 1e20$
Step size collapse	$0 < \alpha < 1e-16$
Invalid step size	$\alpha \leq 0$

4.2 Server Endpoints (server.jl)

GET /optimize

Purpose: Run complete optimization workflow

Query Parameters:

- `function` - Function name (rosenbrock, custom, etc.)
- `custom_formula` - Julia expression if custom function
- `method` - Optimizer (sd, cg, newton, dfp, bfgs, lbfgs)
- `cg_variant` - CG variant (FR, PR, PR_plus)
- `m` - LBFGS memory size
- `linesearch` - Line search (backtracking, gss, dichotomous, quadratic, brent)
- `auto_bracket` - Boolean for automatic bracketing
- `bracket_a`, `bracket_b` - Manual bracket interval
- `x0` - Starting point (comma-separated)
- `dim_x`, `dim_y` - Dimensions for 2D visualization
- `term_criterion` - Termination criterion (gradient, step_size, f_abs, f_rel)
- `tol` - Tolerance value
- `max_iter` - Maximum iterations

Example Response (Success case):

```
{
  "status": "success",
  "iterations": 47,
  "x_hist": [-1.0, -0.95, ..., 1.0],
  "y_hist": [0.0, -0.05, ..., 1.0],
  "full_x_hist": [[-1, 0], [-0.95, -0.05], ..., [1, 1]],
  "f_hist": [24.01, 20.3, ..., 1.2e-6],
  "grad_norm_hist": [24.2, 19.1, ..., 8.3e-5],
  "alpha_hist": [1.0, 0.8, 0.9, ..., 0.05],
  "step_dist_hist": [0.05, 0.08, ..., 0.001],
  "contour_x": [-2, -1.95, ..., 2],
  "contour_y": [-1, -0.95, ..., 3],
  "contour_z": [[24.01, 23.2, ...], [...], ...],
  "true_min_full": [1.0, 1.0],
  "true_min_f": 0.0,
  "diverged": false,
  "divergence_reason": "Success",
  "divergence_iteration": 0,
  "final_grad_norm": 8.3e-5,
```

```

    "final_f_value": 1.2e-6
  }

```

Example Response (Divergence case):

```

{
  "status": "diverged",
  "iterations": 128,
  "diverged": true,
  "divergence_reason": "Gradient norm explosion ( $||\nabla f||=1.2e8$ )",
  "divergence_iteration": 128,
  "final_grad_norm": 1.2e8,
  "final_f_value": 5.4e19
}

```

Example Response (Error case):

```

{
  "status": "error",
  "message": "Function Error: DomainError(-0.5, sqrt will only return a complex result if called with a complex argument)"
}

```

GET /contours

Purpose: Generate contour grid for 2D visualization

Query Parameters:

- **function** - Function name (rosenbrock, ackley, custom, etc.)
- **custom_formula** - Julia expression if custom function
- **xmin, xmax, ymin, ymax** - Plot bounds
- **dim_x, dim_y** - Dimensions to slice
- **x0** - Base coordinates for other dimensions

Example Response:

```

{
  "contour_x": [-2, -1.973, -1.946, ..., 2],
  "contour_y": [-1, -0.974, -0.948, ..., 3],
  "contour_z": [
    [24.01, 21.3, 18.2, ..., null],
    [23.5, 20.8, 17.6, ..., null],
    ...
  ]
}

```

Grid Strategy: 150×150 evaluation points with silent error handling for domain violations (NaN/Inf values replaced with `null`).

Function Dispatch (server.jl)

```
function get_function_objects(selected_function, custom_formula, x0)
    # Returns tuple: (f, ∇f, Hf, error_message)

    if selected_function == "custom"
        # Parse, compile, and differentiate user expression
        f = eval(Meta.parse(formula))
        ∇f = x -> ForwardDiff.gradient(f, x)
        Hf = x -> ForwardDiff.hessian(f, x)

    elseif selected_function in predefined_functions
        # Return precomputed analytical functions
        ...
    end
end
```

5. Frontend System

5.1 HTML Structure (index.html)

The UI is organized into logical panels:

Input Panel

- **Function Selection:** Dropdown with 10 built-in + custom option
- **Custom Formula Input:** Text field with syntax guide
- **Dimensions:** N from 2 to 10
- **Starting Points:** User-defined x_0 inputs
- **2D Plot Configuration:** Axis selection and equal-axes checkbox

Algorithm Configuration Panel

- **Optimization Method:** 6 choices with variants
- **Line Search Method:** 5 choices with bracketing options
- **Termination Criteria:** 4 criteria types
- **Algorithm Parameters:** Tolerance ϵ and max iterations $k_{\max}$

Results Display

- **Main Contour Plot:** Interactive 2D visualization with trajectory
- **Evolution Plots:** 2×2 grid showing:
 - Function value: $f(x_k)$

- Gradient norm: $\|\nabla f(x_k)\|$
- Step size: α_k
- Step distance: $\|x_{k+1} - x_k\|$

Modal Dialogs

- **Info Modal:** Function formulas, syntax guides, termination criteria
- **Preview Modal:** LaTeX rendering of custom functions before running
- **Enlarge Modal:** Full-screen plot expansion

5.2 JavaScript Application (app.js)

State Management

```
// Global state variables
const functionSelect = document.getElementById('functionSelect');
const methodSelect = document.getElementById('methodSelect');
const lsSelect = document.getElementById('lsSelect');
const dimInput = document.getElementById('dimInput');

// Server interaction
const API_BASE_URL = "https://optimization-app-wkcn.onrender.com";
let lastServerInteraction = 0; // Cold start detection
```

Event Listeners

Event	Handler	Purpose
Function change	updateDimensions()	Update UI for selected function
Dimension change	updateDimensions()	Generate x_0 inputs
Method change	Show/hide variants	Conditional UI rendering
Plot pan/zoom	fetchNewContours()	Update contours dynamically
Run button	runOptimization()	Execute optimization request

Key Functions

updateDimensions()

- Detects if function is 2D-only
- Sets dimension constraints
- Generates dimension-dependent UI elements

drawInitialPlot()

- Fetches initial contour grid
- Creates Plotly contour trace

- Attaches relay layout listener for dynamic updates

runOptimization()

- Validates inputs
- Constructs query URL with all parameters
- Shows loading overlay (with cold-start detection)
- Calls `/optimize` endpoint
- Processes response and renders visualizations

juliaToLatex(formula)

- Converts Julia syntax to LaTeX for display
- Handles:
 - Array indexing: $x[1] \rightarrow x_1$
 - Operators: $*$ $\rightarrow \cdot$, $^$ preserved
 - Functions: `sin()`, `cos()`, etc.

Cold Start Handling

```
function showLoading(baseText) {
  const timeSinceLastInteraction = Date.now() - lastServerInteraction;

  if (timeSinceLastInteraction > 600000) { // > 10 minutes
    // After 3.5 seconds, show server wake-up message
    coldStartTimer = setTimeout(() => {
      loadingText.innerHTML = `${baseText}<br><br>
        ☕ Waking up the server... This may take up to 2 minutes.
        Subsequent requests will be instant!`;
    }, 3500);
  }
}
```

Modal Dialogs

Info Modal Content (`modalContent` object):

- **Function Information:** Formulas, global minima, references
- **Termination Criteria:** Mathematical definitions and usage

Plotly Configuration:

- SVG export enabled for all plots
- Responsive sizing with custom margins
- Logarithmic Y-axis support for evolution plots

5.3 Styling (style.css)

Layout System

- **Flexbox-based** responsive grid
- **2-column layout** on desktop, single-column on mobile
- **Grid layout** for 2×2 plot cards

Visual Components

- **Panels:** #f9f9f9 background with subtle borders
 - **Buttons:** Microsoft Fluent Design (blue #0078D7)
 - **Alerts:** Warning boxes with color coding
 - **Loading:** CSS animation spinner with overlay
 - **Formulas:** Monospace code blocks with syntax highlighting
-

6. Optimization Algorithms

6.1 Steepest Descent (SteepestDescent.jl)

Type: First-order method

Convergence Rate: Linear (slow)

Memory: $O(n)$

Algorithm:

```
1. Initialize  $x_0$ 
2. FOR  $k = 0, 1, 2, \dots$ 
   -  $d_k = -\nabla f(x_k)$  [negative gradient]
   -  $\alpha_k = \text{LineSearch}(f, \nabla f, x_k, d_k)$ 
   -  $x_{k+1} = x_k + \alpha_k \cdot d_k$ 
   - Check termination criteria
```

Characteristics:

- ☒ Guaranteed descent at each step
- ☒ Simple and robust
- ☒ Poor performance on ill-conditioned problems (zigzag behavior)
- ☒ Linear convergence rate

Best for: Convex functions, educational purposes

6.2 Conjugate Gradient (ConjugateGradient.jl)

Type: First-order method with memory

Convergence Rate: Superlinear (for quadratic: finite termination in n steps)

Memory: $O(n)$

Variants Available:

Fletcher-Reeves (FR)

$$\beta_k^{\text{FR}} = \frac{\langle \nabla f(x_k), \nabla f(x_k) \rangle}{\langle \nabla f(x_{k-1}), \nabla f(x_{k-1}) \rangle}$$

Characteristics: Theoretically stable, may have slower convergence

Polak-Ribière (PR)

$$\beta_k^{\text{PR}} = \frac{\langle \nabla f(x_k) - \nabla f(x_{k-1}), \nabla f(x_k) \rangle}{\langle \nabla f(x_{k-1}), \nabla f(x_{k-1}) \rangle}$$

Characteristics: Often faster in practice, can diverge on non-convex

Polak-Ribière+ (PR+) - *Default*

$$\beta_k^{\text{PR+}} = \max(0, \beta_k^{\text{PR}})$$

Characteristics: Hybrid approach combining stability of FR with speed of PR

Algorithm (General Schema):

The specific implementation varies by optimizer, but follows this general pattern:

1. Initialize position x_k and gradient ∇f_k
2. Compute search direction (descent direction)
3. Perform line search to find step size α_k
4. Update position and gradient
5. Update method-specific state (Hessian approximations, conjugacy coefficients, etc.)
6. Repeat until convergence

Best for: Medium-scale problems, limited Hessian information

6.3 Newton's Method (NewtonMethod.jl)

Type: Second-order method (exact Hessian)

Convergence Rate: Quadratic (near optimum)

Memory: $O(n^2)$ for Hessian storage

Implementation Details:

- Requires computing the Hessian matrix $\nabla^2 f(x)$ at each iteration
- In this implementation, Hessian is computed analytically via `ForwardDiff.hessian()`
- Uses line search (damping) for global convergence guarantee
- Without damping, Newton's method can diverge on non-convex problems
- Newton direction is computed implicitly through the line search
- Very accurate direction (2nd-order curvature information)
- **Damping:** Uses line search to ensure descent (prevents divergence)
- **Hessian Computation:** Via `ForwardDiff.hessian()`
- **Direction Solving:** Not explicitly shown (implicit via multiply)

Characteristics:

- ☒ Quadratic convergence (very fast near optimum)
- ☒ Precise direction (2nd-order curvature info)
- ☒ Requires computing and inverting $n \times n$ Hessian ($O(n^3)$)
- ☒ Can diverge on non-convex functions without damping
- ☒ Expensive Hessian computation

Best for: Small-dimensional problems (<100 vars), well-conditioned

6.4 Quasi-Newton Methods

DFP (Davidon-Fletcher-Powell) (DFP.jl)

Type: Quasi-Newton (approximate Hessian)

Convergence Rate: Superlinear

Memory: $O(n^2)$

Update Formula:

$$W_{k+1} = W_k + \frac{s_k s_k^T}{\langle s_k, y_k \rangle} - \frac{W_k y_k y_k^T W_k}{\langle y_k, W_k y_k \rangle}$$

where:

Variable	Definition
W_k	inverse Hessian approximation
$s_k = x_{k+1} - x_k$	step
$y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$	gradient difference
$\langle \cdot, \cdot \rangle$	inner product

Implementation (from DFP.jl):

```
function update_approximation!(method::DFPMethod, state::OptimizationState, s, y)
    W = state.inverse_hessian
    ys = dot(y, s)
    yWy = dot(y, W * y)

    if ys > 1e-10 # Strict curvature condition
        term1 = (W * y * transpose(y) * W) / yWy
        term2 = (s * transpose(s)) / ys
        state.inverse_hessian = W - term1 + term2
    end
end
```

Characteristics:

- ☒ Superlinear convergence

- ☒ Only needs gradients (not Hessians)
- ☒ Stable on well-conditioned problems
- ☒ Can be less robust than BFGS
- ☒ Still $O(n^2)$ memory

BFGS (Broyden-Fletcher-Goldfarb-Shanno) (BFGS.jl)

Type: Quasi-Newton (approximate Hessian)

Convergence Rate: Superlinear

Memory: $O(n^2)$

Stability: Better than DFP

BFGS (Broyden-Fletcher-Goldfarb-Shanno) (BFGS.jl)

Type: Quasi-Newton (approximate Hessian)

Convergence Rate: Superlinear

Memory: $O(n^2)$

Stability: Better than DFP

Update Formula:

$$W_{k+1} = V_k W_k V_k^T + \rho_k s_k s_k^T$$

where:

Variable	Definition
$V_k = I - \rho_k (s_k \otimes y_k)$	outer product where $s \otimes y = sy^T$
$\rho_k = 1 / \langle s_k, y_k \rangle$	step-curvature ratio

Implementation (from BFGS.jl):

```
function update_approximation!(method::BFGSMethod, state::OptimizationState, s, y)
    W = state.inverse_hessian
    ys = dot(y, s)

    if ys > 1e-10 # Strict curvature condition
        rho = 1.0 / ys
        I_mat = Matrix{Float64}(I, n, n)
        V = I_mat - rho * (s * transpose(y))
        state.inverse_hessian = V * W * transpose(V) + rho * (s * transpose(s))
    end
end
```

Characteristics:

- ☒ Superlinear convergence
- ☒ Only needs gradients (not Hessians)
- ☒ More stable than DFP

- ☒ Good balance of efficiency and accuracy
- ☒ Still $O(n^2)$ memory
- ☒ Preferred choice over DFP in practice

Best for: Medium-scale problems (up to ~1000 vars)

6.5 L-BFGS (Limited-Memory BFGS) (LBFGS.jl)

Type: Quasi-Newton with limited memory

Convergence Rate: Superlinear

Memory: $O(m \cdot n)$ where $m \ll n$ (typically $m=5-20$)

Direction Computation (Two-Loop Recursion):

The Hessian approximation is not stored explicitly. Instead, the direction is computed on-the-fly from m recent $\{s, y\}$ pairs:

```
# Forward loop: accumulate approximation
q = ∇f(x_k)
for i in [k-m, ..., k-1]
    α_i = ρ_i * ⟨s_i, q⟩
    q = q - α_i * y_i
end

# Diagonal scaling
q = γ * q where γ = ⟨s_{k-1}, y_{k-1}⟩ / ⟨y_{k-1}, y_{k-1}⟩

# Backward loop: construct product
for i in [k-1, ..., k-m]
    β_i = ρ_i * ⟨y_i, q⟩
    q = q + (α_i - β_i) * s_i
end

d_k = -q
```

Characteristics:

- ☒ $O(m \cdot n)$ memory (vs $O(n^2)$ for BFGS)
- ☒ Superlinear convergence
- ☒ Handles large-scale problems well
- ☒ Ground truth in our app: `Optim.jl` attempts L-BFGS first (if it fails, we fall back to Nelder-Mead)
- ☒ Slightly slower convergence than full BFGS with small m
- ☒ Less curvature information with very small m values

Best for: Large-scale problems (> 1000 vars)

7. Line Search Methods

All line search methods solve the 1D minimization problem:

$$\alpha^* = \arg \min_{\alpha > 0} f(x_k + \alpha \cdot d_k)$$

7.1 Backtracking (Inexact) (Backtracking.jl)

Type: Inexact, uses Armijo condition

Computational Cost: Very low

Accuracy: Moderate

Armijo Condition:

$$f(x + \alpha d) \leq f(x) + \beta \cdot \alpha \cdot \nabla f(x)^T d$$

where $\beta \in (0, 1)$ (typically 1e-4)

Algorithm:

1. Start with $\alpha = 1.0$
2. WHILE not (Armijo condition satisfied)
 - $\alpha \leftarrow p \cdot \alpha$ (contraction, $p \approx 0.5$)
3. RETURN α

Parameters:

- $p = 0.5$ (contraction factor)
- $\text{beta} = 1\text{e-}4$ (sufficient decrease)

Implementation (from Backtracking.jl):

```
struct Backtracking <: AbstractLineSearch
    p::Float64      # Contraction factor
    beta::Float64   # Armijo parameter
end

function compute_step_size(ls::Backtracking, f, ∇f, state, d)
    alpha = 1.0
    y_val = f(state.x)
    g = state.gradient

    while f(state.x + alpha * d) > y_val + ls.beta * alpha * dot(g, d)
        alpha *= ls.p
    end

    return alpha
end
```

Characteristics:

- ☒ Fastest (few function evaluations)
- ☒ Guaranteed to find acceptable step
- ☒ Works with any descent direction
- ☒ Not highly accurate
- ☒ May require many optimizer iterations

Best for: Quick iterations, when accuracy not critical

7.2 Golden Section Search (Exact) (GoldenSectionSearch.jl)

Type: Exact bracket-based method

Convergence Rate: linear with a rate of $1/\tau \approx 0.618$

Golden Ratio: $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$

Algorithm (assumes bracket $[a, b]$):

```

1. Initialize:  $[c, d]$  subdivision points
2. REPEAT until  $(b - a) < \epsilon$ 
  - Evaluate  $f(c)$  and  $f(d)$ 
  - If  $f(c) < f(d)$ 
     $b \leftarrow d, d \leftarrow c, c \leftarrow a + (1-\tau) \cdot (b-a)$ 
  - Else
     $a \leftarrow c, c \leftarrow d, d \leftarrow a + \tau \cdot (b-a)$ 
  where  $\tau = 2 - \phi \approx 0.381$ 
3. RETURN  $\alpha = (a+b)/2$ 

```

Bracketing (if auto-bracket enabled):

1. Start with $\alpha_0 = 1.0$
2. Expand exponentially until bracketing triplet (a, b, c) found
3. Refine bracket with golden section

Characteristics:

- ☒ Highly accurate result
- ☒ Deterministic behavior
- ☒ Works on unimodal intervals
- ☒ Requires valid bracket
- ☒ More function evaluations than backtracking

Best for: When line search accuracy matters

7.3 Dichotomous Search (Exact) (DichotomousSearch.jl)

Type: Exact bracket-based method

Convergence Rate: Linear with rate of 0.5

Parameters:

- **tol** - Target interval width (default: 1e-4)
- **delta** - Distinguishability constant for discriminating between nearby points (default: 1e-5)

Algorithm (assumes bracket [a, b]):

```

1. WHILE (b - a) > tol
  - mid ← (a + b) / 2
  - x_minus ← mid - delta
  - x_plus ← mid + delta
  - IF f(x_minus) < f(x_plus)
    b ← x_plus
  - ELSE
    a ← x_minus
2. RETURN α ← (a + b) / 2

```

Implementation Note: The **delta** parameter (distinguishability constant) is crucial for numerical stability. It prevents the algorithm from trying to discriminate between points that are too close given floating-point precision.

Characteristics:

- ☒ Simpler than golden section search
- ☒ Reliable bracketing approach
- ☒ Systematic interval halving
- ☒ Supports custom distinguishability parameter
- ☒ Slower convergence than golden section search
- ☒ Requires careful tuning of **delta** to avoid numerical issues

Best for: Reliable, methodical search with custom precision control

7.4 Quadratic Fit Search (Exact) (QuadraticFitSearch.jl)

Type: Exact model-based method

Convergence Rate: Superlinear (fast when model is accurate)

Function Evaluations: Few (typically 5-10)

Algorithm (assumes bracket [a, c]):

```

1. Evaluate f at three points: a, b=(a+c)/2, c (bracketing triplet)
2. FOR iteration = 1 to max_iter
  - Fit quadratic parabola q(α) through the three points
  - Analytically minimize: α_min from 3-point Lagrange formula
  - Evaluate f(α_min)
  - IF |α_min - α_old| < tol
    RETURN α_min
  - UPDATE three points:

```

```

    Keep the two closest to minimum
    Maintain bracket structure
3. RETURN best  $\alpha$  found

```

Implementation Details (from QuadraticFitSearch.jl):

- Uses Lagrange interpolation to fit quadratic through three points
- Numerator and denominator computed from formula, with safeguard against collinearity
- Automatic fallback if denominator becomes too small

Characteristics:

- ☒ Very fast convergence on smooth functions
- ☒ Uses function curvature information
- ☒ Often fewer evaluations than golden section search
- ☒ Deterministic behavior
- ☒ Can fail if quadratic model is poor
- ☒ Can search for maximum instead of minimum

Best for: Smooth functions with clear quadratic structure

7.5 Brent's Method (Exact) (BrentsMethod.jl)

Type: Exact hybrid method

Convergence Rate: Superlinear

Algorithm (assumes bracket $[a,b]$)

```

1. Initialize:  $x, w, v \leftarrow a + 0.3819660 \cdot (b-a)$  [golden ratio location]
                $f_x, f_w, f_v \leftarrow f(x)$ 
                $d\_step, e \leftarrow 0$  [previous step magnitudes]

2. FOR iteration = 1 to max_iter
   -  $mid \leftarrow (a+b)/2$ 
   -  $tol1 \leftarrow tol \cdot |x| + 1e-10$ 
   -  $tol2 \leftarrow 2 \cdot tol1$ 

   - IF  $|x - mid| \leq (tol2 - (b-a)/2)$ 
     RETURN  $x$  [Interval sufficiently small]

   - IF  $|e| > tol1$ 
     Try PARABOLIC FIT:
     - Compute  $p, q$  from  $(x,w), (x,v)$  points
     - Check if acceptable (within bounds, etc.)
     - IF acceptable:  $d\_step \leftarrow p/q$ 
     - ELSE: Fall back to GOLDEN SECTION step
   - ELSE
     Execute GOLDEN SECTION step

   - Take step:  $\alpha\_new \leftarrow x + d\_step$ 

```

- Evaluate $f(\alpha_{\text{new}})$
- Update $\{x, w, v\}$ and corresponding f -values maintaining bracket and ordering

3. RETURN x

Algorithm Details:

- **Parabolic/Inverse Quadratic Fit:** Fast convergence when function is smooth
- **Golden Section Fallback:** Used if parabolic step is rejected (e.g., goes outside bracket)
- **Bracketing Guarantee:** Always maintains interval $[a,b]$ containing the optimum
- **Numerical Safeguards:** Checks for collinearity, bounds violations, and step size validity

Characteristics:

- ☒ Fastest practical method available
- ☒ Combines speed of interpolation with safety of golden section
- ☒ Robust and reliable on difficult functions
- ☒ Maintains bracketing invariant
- ☒ Industry standard (used in `scipy.optimize`, `Optim.jl`)
- ☒ More complex implementation

Best for: Production-quality code when robustness and speed both matter

7.6 Auto Bracketing

Purpose: Automatically find initial bracket $[a, b]$ for exact line searches

Parameters:

- `initial_step` - Starting step size for first evaluation (default: $1e-4$)
- `expansion` - Multiplicative factor for bracket expansion (default: 2.0)
- `max_iter` - Maximum iterations to find bracket (default: 50)

Algorithm (from `Bracketing.jl`):

```

1. Start at:  $a \leftarrow 0, \quad b \leftarrow a + \text{initial\_step}$ 
2. Evaluate:  $f(a), f(b)$ 

3. IF  $f(b) > f(a)$ 
    Return bracket  $[a, b]$  (minimum already bracketed)
ELSE
    Continue expanding

4.  $c \leftarrow b + \text{expansion} \cdot (b - a)$  [Initial expansion point]
5. FOR iteration = 1 to max_iter
    - Evaluate  $f(c)$ 
    - IF  $f(c) > f(b)$ 
        Return bracket  $[a, c]$  [Bracket found!]
    - ELSE
```

```

Continue expanding:
a ← b, b ← c
c ← b + expansion · (b - a)

```

6. Return last bracket [a, c]

Characteristics:

- ☒ Automatically finds valid bracket for exact line searches
- ☒ Handles cases where initial step is too large or too small
- ☒ No user configuration needed (when using defaults)
- ☒ Can fail if function is monotone (no local minimum in direction)
- ☒ May expand far from starting point on flat functions

Best for: Automatic bracket initialization (default for exact line searches)

8. Test Functions

8.1 N-Dimensional Functions

Rosenbrock (Predefined Analytical)

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2]$$

- **Global Minimum:** $f(\mathbf{1}) = 0$
- **Characteristics:** Long, narrow, parabolic valley
- **Default $\mathbf{x}_0$:** $(-1, 0, 0, \dots, 0)$

Ackley (Via ForwardDiff)

$$f(\mathbf{x}) = -20 \exp \left(-0.2 \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(2\pi x_i) \right) + 20 + e$$

- **Global Minimum:** $f(\mathbf{0}) = 0$
- **Characteristics:** Nearly flat exterior with central hole
- **Difficulty:** Very hard - highly multimodal
- **Interesting Features:** Many local minima, tests global optimization
- **Default $\mathbf{x}_0$:** $(3, 3, 3, \dots, 3)$

Sphere / Axis Parallel Hyper-Ellipsoid (Predefined Analytical)

$$f(\mathbf{x}) = \sum_{i=1}^n i \cdot x_i^2$$

- **Global Minimum:** $f(\mathbf{0}) = 0$
- **Characteristics:** Strictly convex, ill-conditioned (poor scaling)

- **Difficulty:** Easy - convex, but interesting due to conditioning
- **Interesting Features:** Tests algorithm's handling of ill-conditioning
- **Default $\mathbf{x}_0$:** $(3, -4, 3, -4, \dots)$ (alternating)

8.2 2-Dimensional Functions

Himmelblau (Predefined Analytical)

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

- **Global Minima:** Four identical local minima at $f = 0$:
 - $(3.0, 2.0)$
 - $(-2.805, 3.131)$
 - $(-3.779, -3.283)$
 - $(3.584, -1.848)$
- **Characteristics:** Multiple local minima
- **Interesting Features:** Tests multimodal optimization
- **Default $\mathbf{x}_0$:** $(-0.27, -0.9)$

Beale (Via ForwardDiff)

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$$

- **Global Minimum:** $f(3, 0.5) = 0$
- **Characteristics:** Multimodal with sharp peaks, flat valleys
- **Difficulty:** Hard - requires precision
- **Interesting Features:** Large flat regions test gradient magnitudes
- **Default $\mathbf{x}_0$:** $(-1.0, -1.0)$

Booth (Predefined Analytical)

$$f(x, y) = (x + 2y - 7)^2 + (2x + y - 5)^2$$

- **Global Minimum:** $f(1, 3) = 0$
- **Characteristics:** Valley-shaped function
- **Difficulty:** Medium - good test for gradient vs. Hessian methods
- **Interesting Features:** Tests pure quadratic minimization
- **Default $\mathbf{x}_0$:** $(4, 4)$

Goldstein-Price (Via ForwardDiff)

$$f(x, y) = [1 + (x + y + 1)^2(19 - 14x + 3x^2 - 14y + 6xy + 3y^2)] \times \\ [30 + (2x - 3y)^2(18 - 32x + 12x^2 + 48y - 36xy + 27y^2)]$$

- **Global Minimum:** $f(0, -1) = 3$
- **Characteristics:** Complex polynomial, many local minima
- **Difficulty:** Hard - polynomial complexity
- **Interesting Features:** Tests algorithm's ability to escape local minima
- **Default $\mathbf{x}_0$:** $(-2, -2)$

Matyas (Via ForwardDiff)

$$f(x, y) = 0.26(x^2 + y^2) - 0.48xy$$

- **Global Minimum:** $f(0, 0) = 0$
- **Characteristics:** Plate-shaped, no local minima except global
- **Difficulty:** Easy - convex
- **Interesting Features:** Good for learning algorithms
- **Default x_0 :** $(5, 5)$

Lévi N.13 (Via ForwardDiff)

$$f(x, y) = \sin^2(3\pi x) + (x - 1)^2[1 + \sin^2(3\pi y)] + (y - 1)^2[1 + \sin^2(2\pi y)]$$

- **Global Minimum:** $f(1, 1) = 0$
- **Characteristics:** Highly multimodal due to trigonometry
- **Difficulty:** Very hard - oscillating ridges
- **Interesting Features:** Tests algorithms on periodic landscapes
- **Default x_0 :** $(3, 3)$

Three-Hump Camel (Via ForwardDiff)

$$f(x, y) = 2x^2 - 1.05x^4 + \frac{x^6}{6} + xy + y^2$$

- **Global Minimum:** $f(0, 0) = 0$
- **Characteristics:** Three local minima
- **Difficulty:** Medium - multimodal
- **Interesting Features:** Camel-shaped landscape
- **Default x_0 :** $(2, 2)$

8.3 Custom Functions

Syntax: Julia mathematical expressions with `x[1]`, `x[2]`, etc.

Examples:

- `sin(x[1]) + x[2]^2`
- `0.01 * x[1]^2 + x[2]^2`
- `exp(x[1]) * cos(x[2])`

Processing:

1. URL decode formula
2. Remove backslashes (escaping)
3. Parse as Julia expression: `Meta.parse(formula)`
4. Compile to function: `eval(:( $x \rightarrow \$expr$ ))`
5. Differentiate: `ForwardDiff.gradient(f, x)`
6. Verify: evaluate at x_0
7. Return error if any step fails

9. User Guide

9.1 Quick Start

1. **Open Application:** Navigate to deployed URL
2. **Select Function:** Choose from dropdown (Rosenbrock is default)
3. **Set Starting Point:** Modify x_0 values (defaults provided)
4. **Configure Algorithm:**
 - Choose method (default: Steepest Descent)
 - Choose line search (default: Backtracking)
 - Choose termination criterion (default: Gradient magnitude)
5. **Adjust Parameters:**
 - Tolerance: How close to stop (default: $1e-4$)
 - Max iterations: Safety limit (default: 2000)
6. **Run:** Click "Run Optimization"
7. **Analyze Results:** Inspect trajectory and convergence plots

9.2 Custom Function Definition

1. Select "Custom (User Defined)" from function dropdown
2. Enter formula in Julia syntax:

```
sin(x[1]) * exp(x[2]^2)
```

3. Set dimensions ($2 \leq N \leq 10$)
4. Specify starting point
5. Click "Run Optimization" to preview formula
6. Approve and run

Common Mistakes:

- **✗** Using `|x|` instead of `abs(x[1])`
- **✗** Forgetting array indexing: `x[i]` not just `x`
- **✗** Implicit multiplication: `2 x` should be `2*x`
- **✗** Domains that include function singularities

9.3 Interpreting Results

Main Plot

Elements:

- **Contours:** Function level sets (colored by value)
- **Green square:** Starting point x_0
- **Blue dots:** Intermediate iterations
- **Yellow star:** Final converged point
- **Black X:** Ground truth (Optim.jl L-BFGS or Nelder-Mead)
- **Red line:** Optimization trajectory connecting points

Interactions:

- **Hover:** Show point coordinates and function value
- **Zoom:** Click and drag to zoom region
- **Pan:** Hold shift while dragging to pan
- **Download:** Camera icon to save as SVG

Evolution Plots

Function Value $f(\mathbf{x}_k)$:

- ☒ Should monotonically decrease
- **X** If increases: algorithm diverged
- Shape indicates convergence speed

Gradient Norm $\|\nabla f(\mathbf{x}_k)\|$:

- Should approach 0 (indicates convergence)
- Exponential decay = fast convergence
- Linear decay = slow convergence
- Plateau = stalled progress

Step Size α_k :

- Line search parameter
- Should generally decrease
- Very small α ($< 1e-10$) indicates stalling
- $\alpha=1$ most common with Newton

Step Distance $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|$:

- Should approach 0 at convergence
- Similar pattern to gradient norm
- Indicates how far algorithm is moving

Divergence Warning

If algorithm diverges:

- Red warning box appears
- Shows reason (NaN, explosion, collapse)
- Lists iteration where divergence detected
- Shows final gradient norm and function value
- Trajectory plot still shows attempted path

9.4 Line Search Selection Guide

Criterion	Recommended	Trade-off
Speed Priority	Backtracking	Few evals, less accurate

Criterion	Recommended	Trade-off
Accuracy Priority	Brent's Method	More evals, high precision
Safety/Reliability	Golden Section	Deterministic, moderate speed
Production Code	Brent's Method	Industry standard
Learning/Visual	GSS or Dichotomous	Clear behavior

9.5 Parameter Tuning

Tolerance (ϵ)

Default: 1e-4

- **1e-2**: Very loose, fast (educational purposes)
- **1e-4**: Standard, good balance
- **1e-6**: Tight, more iterations
- **1e-10**: Very tight, expensive, risk of numerical issues

Recommendation: Start at 1e-4, adjust based on convergence plot

Max Iterations (**k_max**)

Default: 2000

- **100**: Fast convergence (will hit limit on hard problems)
- **2000**: Standard, handles most problems
- **10000**: For very difficult or high-dimensional problems

Recommendation: Use default, increase only if hitting limit regularly

CG Variant

- **PR+** (Default): Safest, combines strengths
- **PR**: Fastest but risky on non-convex
- **FR**: Most stable but sometimes slower

L-BFGS Memory Size (**m**)

Default: 5

- **m=1**: Minimal memory, BFGS-like but limited
- **m=5**: Standard, good balance
- **m=20**: More curvature info, more memory

10. API Reference

Backend REST API

Base URL

```
https://optimization-app-wkcn.onrender.com
```

Endpoint: /optimize

```
GET /optimize?  
function=rosenbrock&method=sd&linesearch=backtracking&x0=-1.0,0.0&...
```

Full Query String Format:

```
?function=<string>  
&custom_formula=<url-encoded-expression>  
&method=<sd|cg|newton|dfp|bfgs|lbfgs>  
&cg_variant=<FR|PR|PR_plus>  
&m=<integer>  
&linesearch=<backtracking|gss|dichotomous|quadratic|brent>  
&auto_bracket=<true|false>  
&bracket_a=<float>  
&bracket_b=<float>  
&x0=<comma-separated-floats>  
&dim_x=<int>  
&dim_y=<int>  
&term_criterion=<gradient|step_size|f_abs|f_rel>  
&tol=<float>  
&max_iter=<int>
```

Response (Success):

```
{  
  "alpha_hist": [1.0, 0.25, 0.03125, ...],  
  "contour_x": [-6.0, -5.919, -5.839, ..., 2.0],  
  "contour_y": [-8.0, -7.855, -7.711, ..., 3.0],  
  "contour_z": [[98.5, 87.3, ...], [...], ...],  
  "diverged": false,  
  "divergence_iteration": 0,  
  "divergence_reason": "Success",  
  "f_hist": [181.6, 179.1, 35.0, ..., 2.2e-11],  
  "final_f_value": 0.0,  
  "final_grad_norm": 1.2e-5,  
  "full_x_hist": [[-0.27, -0.9], [-0.19, -0.95], ..., [3.0, 2.0]],  
  "grad_norm_hist": [24.2, 19.1, 14.5, ..., 1.2e-5],  
  "iterations": 19,  
  "status": "success",  
  "step_dist_hist": [0.15, 0.08, 0.12, ..., 0.001],  
}
```

```
"true_min_f": 0.0,  
"true_min_full": [3.0, 2.0],  
"x_hist": [-0.27, -0.12, ..., 3.00],  
"y_hist": [-0.9, -0.50..., 2.00]  
}
```

Response (Divergence):

```
{  
  "alpha_hist": [1.0000173070317, 1.0000173070317],  
  "contour_x": [-8.36e+31, -8.30e+31, ..., 7.60e+30],  
  "contour_y": [-1.11e+32, -1.10e+32, ..., 1.01e+31],  
  "contour_z": [[98.5, 87.3, ...], [...], ...],  
  "diverged": true,  
  "divergence_iteration": 2,  
  "divergence_reason": "Function value explosion (f=1.27e32)",  
  "f_hist": [2.5, 11259193928280058, 1.267e+32],  
  "final_f_value": 1.267e+32,  
  "final_grad_norm": 1,  
  "full_x_hist": [[1.5, 2], [6755516356968036, 9007355142624046],  
  [-7.606035236984129e+31, -1.0141380315978837e+32]],  
  "grad_norm_hist": [1,1,1],  
  "iterations": 2,  
  "status": "diverged",  
  "step_dist_hist": [11259193928280056, 1.267e+32],  
  "true_min_f": 4.57e-9,  
  "true_min_full": [-3.66e-9, 2.73e-9],  
  "x_hist": [1.5, 6755516356968036, -7.60e+31],  
  "y_hist": [2, 9007355142624046, -1.01e+32]  
}
```

Response (Error):

```
{  
  "error": "Function Error: DomainError(-0.5, sqrt will only return a complex  
  result if called with a complex argument)"  
}
```

Endpoint: /contours

```
GET /contours?function=rosenbrock&xmin=-2&xmax=2&ymin=-1&ymax=3&...
```

Query Parameters:

```
?function=<string>
&custom_formula=<url-encoded>
&xmin=<float>
&xmax=<float>
&ymin=<float>
&ymax=<float>
&dim_x=<int>
&dim_y=<int>
&x0=<comma-separated-floats>
```

Response:

```
{
  "contour_x": [-2, -1.973, -1.946, ..., 2],
  "contour_y": [-1, -0.974, -0.948, ..., 3],
  "contour_z": [
    [24.01, 21.3, 18.2, ..., null],
    [23.5, 20.8, 17.6, ..., null],
    ...
  ]
}
```

Grid Details:

- Resolution: 150×150
- Order: Row-major (row varies faster)
- NaN/Inf handling: Replaced with `null`
- Domain errors: Silently omitted

11. Infrastructure & Cold Start Handling

Cold Start Handling

Issue: Free tier servers sleep after 15 minutes of inactivity

Solution: Frontend implements cold-start detection

```
if (timeSinceLastInteraction > 600000) { // > 10 minutes
  // After 3.5 seconds, show message about server wake-up
  // Expected time: up to 2 minutes
}
```

User Experience:

- First request after inactivity: 30-120 seconds
- Subsequent requests: < 1 second

- Message informs user of delay

12. File Structure Summary

```
Chapter_04/
├── backend/
│   ├── server.jl           # Main HTTP server, endpoints
│   ├── Core.jl            # Optimization framework
│   ├── TestFunctions.jl   # 10 test functions
│   ├── Project.toml       # Julia dependencies
│   ├── Manifest.toml      # Dependency lock
│   ├── Dockerfile         # Container definition
│   └── Optimizers/
│       ├── SteepestDescent.jl # First-order method
│       ├── ConjugateGradient.jl # CG (FR, PR, PR+)
│       ├── NewtonMethod.jl    # Newton with Hessian
│       ├── DFP.jl            # Quasi-Newton (old)
│       ├── BFGS.jl          # Quasi-Newton (modern)
│       └── LBFGS.jl         # Limited-memory BFGS
│   └── LineSearch/
│       ├── Backtracking.jl    # Inexact Armijo
│       ├── GoldenSectionSearch.jl # Golden ratio bisection
│       ├── DichotomousSearch.jl # Interval halving
│       ├── QuadraticFitSearch.jl # Quadratic model
│       ├── BrentsMethod.jl    # Hybrid interpolation
│       └── Bracketing.jl      # Auto-bracketing utility
├── frontend/
│   ├── index.html         # UI structure + modals
│   ├── app.js             # Event handlers, API calls
│   └── style.css           # Responsive styling
└── DOCUMENTATION.md      # This file
```

13. Troubleshooting

Backend Issues

Problem	Cause	Solution
"Module not found"	Dependencies missing	Run <code>Pkg.instantiate()</code>
Port already in use	Another process on :8080	Kill process or change port
Slow first request	Cold start (production)	Wait 30-120s, then instant
Custom function fails	Syntax error in formula	Check Julia math syntax
Divergence detected	Poor starting point	Try different x_0

Frontend Issues

Problem	Cause	Solution
"CORS error"	Backend unreachable	Check API_BASE_URL config
Plot not rendering	Plotly CDN down	Check browser console
Formula preview blank	MathJax not loaded	Refresh page
Contours not updating	Network issue	Check network tab

Algorithm Issues

Problem	Cause	Solution
Not converging	Tolerance too tight	Increase ϵ or k_{max}
Zigzag trajectory	Algorithm is Steepest Descent	Try Newton or BFGS
Divergence at iteration 1	Invalid starting point	Check function domain
Very slow convergence	Ill-conditioned problem	Try different algorithm

14. References & Resources

Software Documentation

- [Julia Language](#)
- [ForwardDiff.jl](#)
- [Optim.jl](#)
- [Oxygen.jl](#)
- [Plotly.js](#)

Online Resources

- [SFU Test Functions Library](#)
- [MathJax Documentation](#)
- [Render.com Deployment Guides](#)

Document Version: 2.1 **Last Updated:** April 29, 2026 **Author:** Tomáš Halmazňa